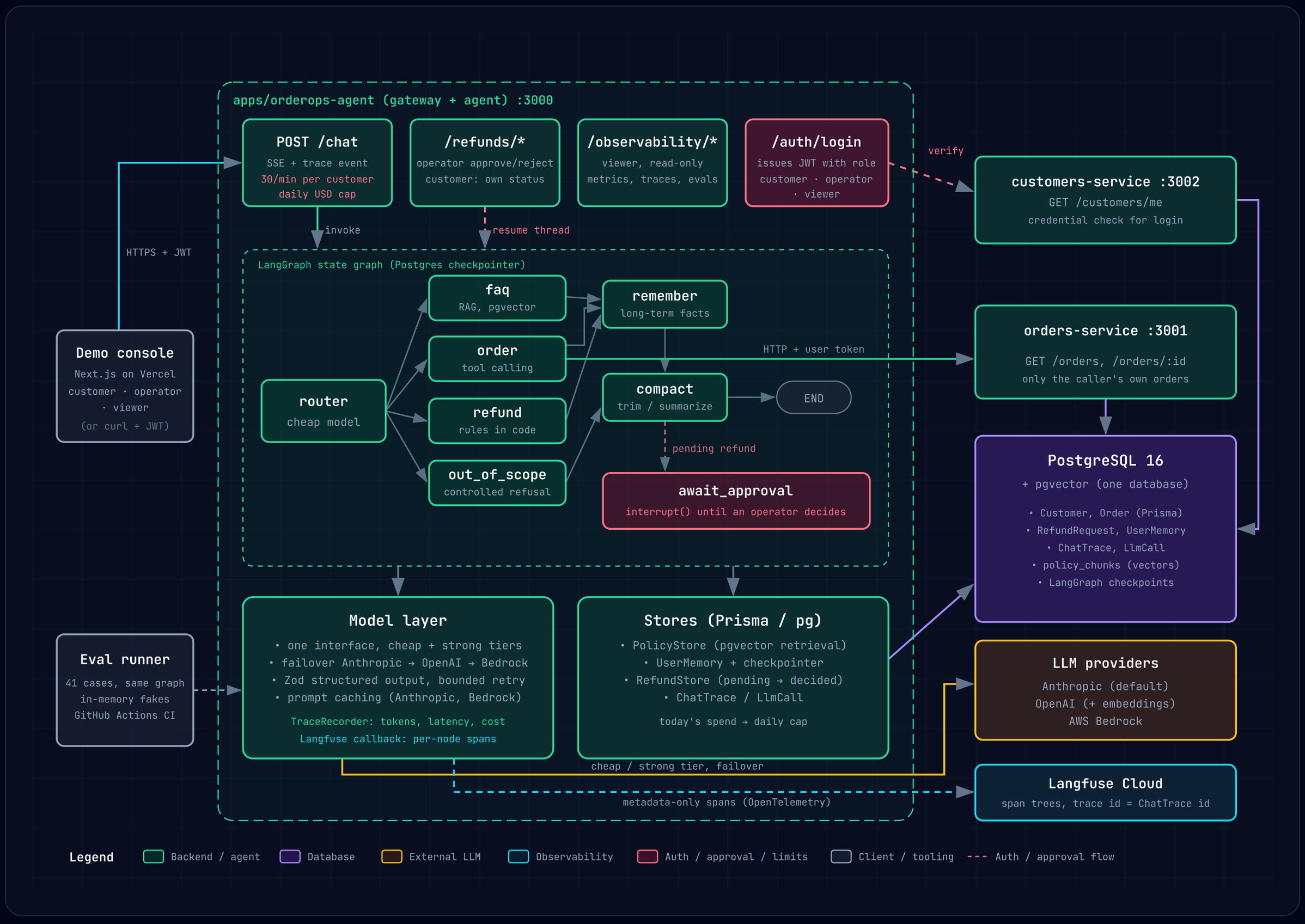
OrderOps Agent Architecture

NestJS gateway + LangGraph agent, two mock microservices, Postgres + pgvector, three LLM providers, Next.js demo console



Request flow

- The console's server logs in at /auth/login; the gateway checks the password with customers-service and signs a JWT with the role (kept in an httpOnly cookie)
- POST /chat checks the rate limit and the daily USD cap, then streams SSE: token, tool_call, approval_required, final, then trace
- router picks faq, order, refund or out_of_scope; then remember and compact
- order tools call orders-service with the caller's token

Guardrails

- Refund rules enforced in code: delivered, 30 days, amount ≤ total, one open request
- Zod validation with bounded retry and deterministic fallback
- interrupt() pauses the thread until an operator approves or rejects; operators cannot decide their own request
- Public demo: 30 chats per minute per customer, per-IP limits on the console, daily USD cap with a fixed reply

Models and observability

- Cheap tier classifies, strong tier answers; failover across 3 providers
- Every request writes ChatTrace + LlmCall (tokens, latency, USD cost) and sends metadata-only spans to Langfuse
- /observability (viewer role) feeds the console dashboard: latency by route, cost, traces, eval scorecard
- Evals: 41 cases, route / tool / trajectory / guardrail metrics, optional LLM judge; keyless PR gate, weekly live run with a cost cap